

CSRF — Token Validation Depends on Request Method

Lab

PortSwigger Web Security Academy — CSRF where token validation depends on request method

Vulnerability

Cross-Site Request Forgery (CSRF)

Severity

Medium

Description

The application implements CSRF token validation inconsistently depending on the HTTP request method.

The application properly validates the CSRF token for one request method, but the validation can be bypassed by changing the request method. As a result, an attacker can construct a malicious request that performs a state-changing action on behalf of an authenticated victim.

Because the victim's browser automatically includes their session cookies with the forged request, the application processes the action as if it were intentionally performed by the victim.

Lab Objective

The objective of this lab is to exploit the inconsistent CSRF token validation and perform the intended state-changing action without the victim intentionally submitting the request.

Setup

- Platform: PortSwigger Web Security Academy
- Vulnerability: CSRF

- Tool: Burp Suite
- Browser: Any Burp-compatible browser
- Authentication: Authenticated victim session

Exploitation Steps

1. Identify the vulnerable request

Log in to the application and locate the functionality that changes account information.

Capture the request using Burp Suite Proxy.

The request contains a CSRF token and uses a specific HTTP method.

2. Test CSRF token validation

Send the request to Burp Repeater and modify the CSRF token.

Observe that the application rejects the request when the token is invalid.

This indicates that token validation is present for the original request method.

3. Change the HTTP method

Change the request method while keeping the same parameters.

For example, convert the request from:

POST /my-account/change-email HTTP/1.1

to an alternative supported method.

The application processes the request without enforcing the same CSRF-token validation.

4. Construct the CSRF exploit

The attacker can create a malicious request that causes the victim's browser to send the state-changing request.

Because the victim is already authenticated, their session cookie is automatically included.

5. Verify the impact

After the forged request is processed, verify that the victim's account information has been changed.

This confirms that the CSRF protection can be bypassed through HTTP-method manipulation.

Impact

An attacker could potentially perform unauthorized actions using the privileges of an authenticated victim.

Depending on the affected functionality, this may allow:

- Changing account information
- Modifying security settings
- Performing unauthorized transactions
- Changing an email address
- Triggering other state-changing operations

The actual impact depends on which application functionality is vulnerable.

Root Cause

The primary issue is **inconsistent CSRF-token validation across HTTP request methods**.

The application treats different request methods differently instead of applying the same CSRF protection to every state-changing operation.

OWASP Mapping

OWASP Top 10: A01 — Broken Access Control

CSRF can result in unauthorized actions being performed using an authenticated user's privileges.

CWE-352 — Cross-Site Request Forgery (CSRF)

Remediation

1. Validate CSRF tokens consistently for every state-changing HTTP method.
2. Do not rely on the HTTP request method as a CSRF security boundary.
3. Use framework-provided CSRF protection where possible.
4. Use unpredictable, session-bound CSRF tokens.

5. Consider implementing appropriate **SameSite** cookie attributes as an additional defense.
6. Validate the **Origin** or **Referer** header where appropriate as defense-in-depth.
7. Avoid allowing state-changing actions through unintended HTTP methods.

Conclusion

The application contains a CSRF vulnerability because CSRF-token validation depends on the HTTP request method. By changing the request method, an attacker can bypass the intended token validation and cause a state-changing request to be executed using an authenticated victim's session.

The vulnerability can be mitigated by applying consistent CSRF protection to all state-changing requests.

Tools Used

- Burp Suite
- Web Browser
- PortSwigger Web Security Academy